BRACT’s Vishwakarma Institute of Technology, Pune

Practical No. 3 — Deep Learning

Department	CSE-AI	**Semester**	5
Academic Year	2026-27	**Division**	E
Batch	3	**Roll No.**	64
PRN	12412320	**Name**	Samyak Shende
Subject	Deep Learning		

BRACT’s

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 3

Roll no: 64 PRN No: 12412320 Name of Student: Samyak Shende

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

Import the required libraries: Pandas, NumPy, TensorFlow/Keras, Matplotlib, and Scikit-learn.

Load the Bank Marketing dataset from the uploaded CSV file.

Separate the input features and target variable deposit.

Convert the target values from yes/no to 1/0.

Convert categorical attributes into numerical values using one-hot encoding.

Divide the dataset into 80% training data and 20% testing data.

Standardize the input features using StandardScaler.

Create a feed-forward neural network containing:

Input layer

32-neuron hidden layer with ReLU

16-neuron hidden layer with ReLU

1-neuron output layer with Sigmoid

Perform forward propagation through the network to generate predictions.

Calculate the prediction error using binary cross-entropy loss.

Perform backpropagation to calculate gradients and update weights using the Adam optimizer.

Train separate models using learning rates 0.0001, 0.001, 0.01, and 0.1, keeping epochs constant.

Compare the validation accuracy obtained for different learning rates.

Train separate models for 10, 25, 50, and 100 epochs using a fixed learning rate.

Compare model accuracy for different epoch values.

Evaluate the final model on the test dataset.

Generate a confusion matrix to analyze classification performance.

Conclude how learning rate and number of epochs affect model performance.

Code :

```
# ============================================================
# Forward & Backpropagation using TensorFlow/Keras
# Dataset: Bank Marketing
# ============================================================
# 1. IMPORT LIBRARIES
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
# ============================================================
# 2. UPLOAD AND LOAD DATASET
# ============================================================
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print("Dataset Shape:", df.shape)
display(df.head())
# ============================================================
# 3. PREPROCESS DATA
# ============================================================
# Separate features and target
X = df.drop("deposit", axis=1)
# yes -> 1, no -> 0
y = df["deposit"].map({
"no": 0,
"yes": 1
})
# Convert categorical features into numerical features
X = pd.get_dummies(
X,
drop_first=True,
dtype=int
)
print("Features after encoding:", X.shape[1])
```

```
# ============================================================
# 4. TRAIN-TEST SPLIT
# ============================================================
X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.2,
random_state=42,
stratify=y
)
# ============================================================
# 5. FEATURE SCALING
# ============================================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# ============================================================
# 6. CREATE NEURAL NETWORK
# ============================================================
def create_model(learning_rate=0.001):
model = tf.keras.Sequential([
tf.keras.layers.Input(shape=(X_train.shape[1],)),
tf.keras.layers.Dense(
32,
activation="relu"
),
tf.keras.layers.Dense(
16,
activation="relu"
),
tf.keras.layers.Dense(
1,
activation="sigmoid"
)
])
model.compile(
optimizer=tf.keras.optimizers.Adam(
learning_rate=learning_rate
```

```
),
loss="binary_crossentropy",
metrics=["accuracy"]
)
return model
model = create_model()
model.summary()
# ============================================================
# 7. FORWARD PROPAGATION
# ============================================================
sample = X_train[:1]
prediction = model(
sample,
training=False
)
print(
"\nForward Propagation Output:",
prediction.numpy()[0][0]
)
# ============================================================
# 8. BACKPROPAGATION DEMONSTRATION
# ============================================================
x_batch = tf.convert_to_tensor(
X_train[:32],
dtype=tf.float32
)
y_batch = tf.convert_to_tensor(
y_train.iloc[:32].values.reshape(-1, 1),
dtype=tf.float32
)
loss_function = tf.keras.losses.BinaryCrossentropy()
with tf.GradientTape() as tape:
# Forward propagation
predictions = model(
x_batch,
training=True
)
loss = loss_function(
y_batch,
```

predictions

)

Backpropagation

gradients = tape.gradient(

loss,

model.trainable_variables

)

Update weights

model.optimizer.apply_gradients(

zip(

gradients,

model.trainable_variables

)

)

print("Loss:", loss.numpy())

 # ==

 # 9. LEARNING RATE EXPERIMENT

 # ==

learning_rates = [

0.0001,

0.001,

0.01,

0.1

]

lr_accuracy = []

for lr in learning_rates:

tf.keras.backend.clear_session()

model = create_model(lr)

history = model.fit(

X_train,

y_train,

validation_split=0.2,

epochs=30,

batch_size=32,

verbose=0

)

accuracy = history.history[

"val_accuracy"

][-1]

```
lr_accuracy.append(accuracy)
print(
"Learning Rate:",
lr,
"Validation Accuracy:",
round(accuracy, 4)
)
# Plot
plt.plot(
learning_rates,
lr_accuracy,
marker="o"
)
plt.xscale("log")
plt.xlabel("Learning Rate")
plt.ylabel("Validation Accuracy")
plt.title(
"Effect of Learning Rate on Model Performance"
)
plt.grid(True)
plt.show()
# ============================================================
# 10. EPOCH EXPERIMENT
# ============================================================
epoch_values = [
10,
25,
50,
100
]
epoch_accuracy = []
for epochs in epoch_values:
tf.keras.backend.clear_session()
model = create_model(
learning_rate=0.001
)
history = model.fit(
X_train,
y_train,
```

```
validation_split=0.2,
epochs=epochs,
batch_size=32,
verbose=0
)
accuracy = history.history[
"val_accuracy"
][-1]
epoch_accuracy.append(accuracy)
print(
"Epochs:",
epochs,
"Validation Accuracy:",
round(accuracy, 4)
)
# Plot
plt.plot(
epoch_values,
epoch_accuracy,
marker="o"
)
plt.xlabel("Number of Epochs")
plt.ylabel("Validation Accuracy")
plt.title(
"Effect of Epochs on Model Performance"
)
plt.grid(True)
plt.show()
# ============================================================
# 11. FINAL MODEL
# ============================================================
final_model = create_model(
learning_rate=0.001
)
history = final_model.fit(
X_train,
y_train,
validation_split=0.2,
epochs=50,
batch_size=32,
```

```
verbose=0
)
test_loss, test_accuracy = final_model.evaluate(
X_test,
y_test,
verbose=0
)
print(
"\nFinal Test Accuracy:",
round(test_accuracy, 4)
)
print(
"Final Test Loss:",
round(test_loss, 4)
)
# ============================================================
# 12. CONFUSION MATRIX
# ============================================================
y_prob = final_model.predict(
X_test,
verbose=0
)
y_pred = (
y_prob.flatten() >= 0.5
).astype(int)
cm = confusion_matrix(
y_test,
y_pred
)
ConfusionMatrixDisplay(
confusion_matrix=cm,
display_labels=["No", "Yes"]
).plot()
plt.title("Confusion Matrix")
plt.show()
```

Output :

Experiment Summary

Dataset	Bank Marketing dataset
Target	deposit

Data split	80% training / 20% testing
Preprocessing	One-hot encoding and StandardScaler
Architecture	32 ReLU $\rightarrow$ 16 ReLU $\rightarrow$ 1 Sigmoid
Optimizer	Adam
Learning rates	0.0001, 0.001, 0.01, 0.1
Epochs	10, 25, 50, 100
Evaluation	Confusion matrix; analyze learning rates and epochs